

TENSORKIT • BOOK ONE OF FOUR

How Automatic Differentiation Works

Building a deep learning framework's engine from nothing but arrays — the tape, the graph walk, and why the chain rule runs backwards.

Harshit Agrawal

github.com/harshitagrwal20/tensorkit

Written to be read start to finish. No prior knowledge assumed beyond Python.

Contents

1. The knob problem

- 1.1 The scale of what is being asked
- 1.2 What TensorKit is

2. Three ways to get a derivative

- 2.1 Symbolic differentiation
- 2.2 Numerical differentiation
- 2.3 Automatic differentiation
- 2.4 Forward mode and reverse mode

3. The chain rule, run backwards

- 3.1 A worked example, by hand
- 3.2 Vectors, and the Jacobian that must not be built
- 3.3 The vector–Jacobian product

4. The tape

- 4.1 What a node holds
- 4.2 Building a graph by hand
- 4.3 What is not on the tape

5. The order of the walk

- 5.1 The obvious approach, and how it fails
- 5.2 The fix: linearise first, then walk
- 5.3 Seeding

6. Accumulation

- 6.1 The smallest possible case
- 6.2 Why the sum is the right answer
- 6.3 Why this is the bug that costs a week
- 6.4 What TensorKit does about it

7. Broadcasting, and its inverse

- 7.1 What broadcasting does
- 7.2 What the backward pass has to do
- 7.3 The rule, exactly
- 7.4 The deeper framing: unbroadcast is an adjoint
- 7.5 The dual direction: sum's backward

8. Checking the gradients

- 8.1 Central differences, and why they are worth twice the cost
- 8.2 Choosing h
- 8.3 Why float32 cannot be used, with the arithmetic
- 8.4 Relative error, not absolute
- 8.5 Reading a failure
- 8.6 Does the oracle have teeth?
- 8.7 Kinks

9. The gradients worth deriving by hand

- 9.1 Matmul
- 9.2 Softmax
- 9.3 Log-sum-exp, and why cross-entropy is fused
- 9.4 LayerNorm, and why it has three terms

10. What you have built, and what you have not

- 10.1 What you have
- 10.2 What you have not
- 10.3 Where this goes next

The knob problem

You have a function with a number in it, and the number is wrong.

That is the whole of machine learning, stated unkindly. A network is a function from an input to a prediction. Somewhere inside it are several million numbers — weights and biases — and a separate function, the loss, that scores how badly the prediction missed. Training means turning those millions of knobs until the score gets small.

Start with one knob. Suppose a single weight w sits inside a network, the loss is currently 2.31, and you want to know what to do with w . There is an obvious experiment: nudge it and see.

```
w = 0.400 → loss = 2.3100
w = 0.401 → loss = 2.3094
```

The loss fell by 0.0006 when w rose by 0.001. So the loss is falling at a rate of about -0.6 per unit of w . Increase w . That number, -0.6 , is the derivative $\partial \text{loss} / \partial w$, and it is the only thing about w you actually needed to know.

Notice what the derivative gives you that the raw loss does not. The loss says *how bad*. The derivative says *which way, and how urgently*. It has a sign, which points downhill, and a magnitude, which says how much the loss cares. A weight with a derivative of -0.6 matters six hundred times more, locally, than one with a derivative of -0.001 . Gradient descent is one line:

```
for each parameter w:
    w ← w - learning_rate × ∂loss/∂w
```

Everything else in an optimiser — momentum, Adam's running moments, weight decay — is decoration on that line. The line is useless without the partial derivatives, and it needs one *per parameter*.

The scale of what is being asked

That last sentence is where the problem lives. Not "a derivative" — a derivative for every knob, recomputed from scratch every single training step, because the derivative at $w = 0.400$ tells you nothing about the derivative at $w = 0.617$.

Put numbers on it. A small convolutional network for MNIST has around 100,000 parameters. A transformer that can hold a conversation has 10^{10} or more. Training runs for tens of thousands of steps. So the machinery that produces gradients runs millions of times, and each run must produce millions of numbers. Whatever that machinery is, it has to be *fast* — not "fast enough for a demo", fast in the sense of costing about the same as running the network forwards.

THE KEY IDEA

The problem is not "compute a derivative". It is "compute $\partial \text{loss} / \partial w$ for every one of n parameters, where n is 10^5 to 10^{11} , at a total cost comparable to one forward pass". That constraint — one scalar output, enormous numbers of inputs, and a hard cost budget — determines every design decision in the rest of this book.

What TensorKit is

TensorKit is a deep learning framework built from NumPy arrays and nothing else. No PyTorch, no JAX, no autodiff package anywhere in the dependency tree; a CI job enforces that. It has two halves. The engine — a tensor type that records what happens to it, and a graph walk that turns those records into gradients — is the subject of this book. The library on top of the engine — layers, losses, optimisers, a convolution — is ordinary code once the engine exists.

The target is a convolutional network trained on MNIST to over 98% test accuracy, where every gradient the engine computed has been checked against numerical differentiation to a stated tolerance. That last clause is not a formality. Almost every mistake in this subject is *silent*: the code runs, the loss goes down, and the model is quietly worse than it should be. A recurring theme of what follows is that you cannot trust a gradient you have not checked.

WORTH KNOWING

Notation, once. $\partial \text{loss} / \partial w$ and "the gradient of w " mean the same thing throughout, and in code that number lives in `w.grad`. For an intermediate value rather than a parameter the traditional name is the *adjoint*, written $\bar{w}$. This book says "gradient" for both.

Exercises

1. A weight has gradient `+3.2` and the learning rate is `0.01`. What is the new weight, and did the loss go up or down as a result of the change?
2. Why does the gradient have to be recomputed every step? Give a function of one variable whose derivative at `x = 1` tells you nothing useful about its derivative at `x = 2`.
3. A network has 100,000 parameters and trains for 10,000 steps. How many individual partial derivatives does the whole training run consume? Give the number.
4. Gradient descent needs the derivative, but it discards everything else about the loss surface — curvature, for instance. Name a situation where a purely first-order step goes badly, and say what a second-order method would have known that gradient descent did not.

Three ways to get a derivative

There are exactly three ways a computer can obtain a derivative, and it is worth being precise about all three, because two of them are used constantly in the wrong place.

Symbolic differentiation

Treat the function as an algebraic expression and apply the rules of calculus to produce a new expression.

$d/dx (x^2 \sin x) = 2x \sin x + x^2 \cos x$. This is what Mathematica does, and it is exact.

It also explodes. Every product rule turns one node into two; every quotient rule into three, and shared subterms get written out once per appearance rather than computed once. Stack that thirty layers deep and the derivative is an expression with a number of terms exponential in the depth. Correct, and unusable. The technical name is *expression swell*.

There is a second, quieter problem: a neural network is not an algebraic expression. It contains `if` statements, loops whose trip count depends on the data, and indexing. Symbolic differentiation needs a closed form. Real code often does not have one.

Numerical differentiation

Go back to the experiment from the first chapter. Perturb the input, rerun the function, divide.

$$\partial f / \partial x \approx (f(x + h) - f(x)) / h$$

This is the *forward difference*. It needs no calculus, works on any function you can call, and is trivially correct to implement. It also has two fatal problems as an engine.

The first is accuracy. The approximation is only exact in the limit as $h \rightarrow 0$, and you cannot take that limit in floating point. Chapter 8 works this out in detail; the short version is that the best you can do is about ten significant digits, and only with care.

The second is cost, and it is the one that kills it. $f(x + h)$ perturbs *one* input. To get the partial derivative with respect to each of `n` parameters you must perturb each of them separately, which is `n` forward passes.

Do the arithmetic for the MNIST network. About 100,000 parameters; central differences — the accurate variant, evaluating at $x + h$ and $x - h$ — need two forward passes each, so 200,000 forward passes for one gradient. At 10 milliseconds per forward pass on a batch, that is 2,000 seconds, or 33 minutes, for a single optimiser step. An epoch is 469 steps: roughly six months.

Reverse mode gets the same 100,000 numbers in about $2\times$ the cost of one forward pass — call it three forward-pass-equivalents including the forward pass itself. The ratio is 200,000 to 3: about **70,000× less work**, and the gap grows linearly with the parameter count.

THE KEY IDEA

Finite differences cost one pair of forward passes *per parameter*. Reverse mode costs a constant multiple of one forward pass *in total*. That is not a constant-factor difference; it is the difference between $O(n \cdot \text{forward})$ and $O(\text{forward})$.

Finite differences survive in TensorKit — as the test oracle in `gradcheck.py`, applied to tiny tensors, never as the engine. That is exactly the right role for them: too slow to compute gradients with, perfect for checking gradients computed some other way.

Automatic differentiation

The third way starts from an observation. Any program, however complicated, is ultimately a sequence of *primitive* operations — add, multiply, exponentiate, matrix-multiply — and you know the derivative of every primitive in closed form. The chain rule composes them. So you do not need the derivative of the whole function symbolically; you need to apply the chain rule numerically, step by step, at the specific point you care about.

Automatic differentiation is exact to floating-point precision, like symbolic differentiation, and it works on arbitrary control flow, like numerical differentiation. It gets both because it never builds an expression for the derivative — it just evaluates one. There are two ways to run it, and the choice between them is the most important idea in this chapter.

Forward mode and reverse mode

The chain rule for $y = f(g(x))$ is $dy/dx = (dy/dg) \cdot (dg/dx)$. For a longer chain, $x \rightarrow a \rightarrow b \rightarrow c \rightarrow y$, it is a product of four factors:

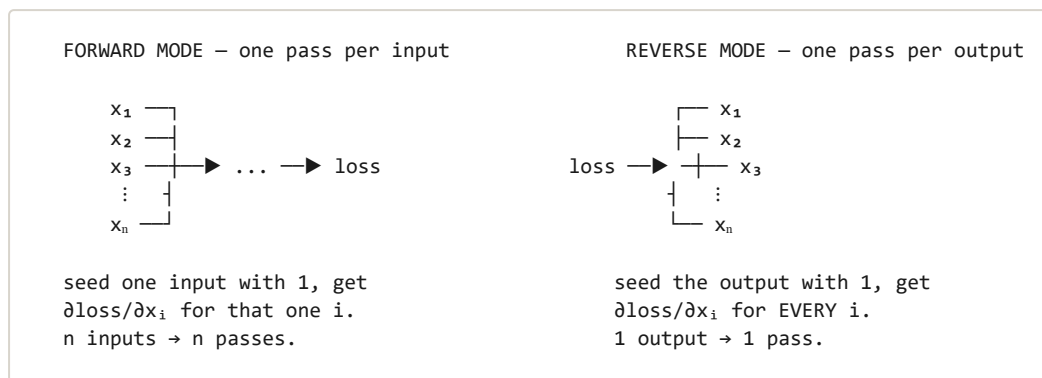
$$dy/dx = dy/dc \cdot dc/db \cdot db/da \cdot da/dx$$

Matrix multiplication is associative, so you may bracket that product however you like. There are two natural choices.

Forward mode brackets from the right: $((dy/dc \cdot dc/db) \cdot db/da) \cdot da/dx$ evaluated right-to-left, starting from da/dx . It runs alongside the forward pass. Every value carries a companion — "and here is my derivative with respect to x " — computed at the same time.

Reverse mode brackets from the left, starting with dy/dc . It runs backwards from the output, and it cannot start until the forward pass has finished, because it needs the intermediate values.

Both are exact. Both cost about the same per pass. The difference is *what one pass gives you*.



The asymmetry that decided the field. Forward mode's cost scales with the number of inputs; reverse mode's with the number of outputs.

A forward-mode pass computes the derivative of *everything* with respect to *one* input. To get all n partials, run it n times. That is finite differences' cost profile, only exact.

A reverse-mode pass computes the derivative of *one* output with respect to *everything*. To get partials for all m outputs, run it m times.

Now look at machine learning. The loss is a single scalar. $m = 1$. The parameters number in the millions or billions. n is huge. Reverse mode needs one pass; forward mode needs n . The whole field runs on reverse mode for that reason and no other.

ASIDE

Forward mode is *better* whenever the shape is reversed — the sensitivity of a whole simulation to a handful of physical constants, say. It also needs no tape, since the derivative is computed alongside the value and thrown away with it, which is why JAX offers both and why forward-over-reverse is the standard trick for Hessian-vector products. TensorKit builds reverse mode only; forward mode is a documented non-goal.

COMMON MISTAKE

The most common confusion here is to think reverse mode is "the fast one". It is not faster per pass — it is often slightly slower, and it uses far more memory because it has to keep every intermediate value alive until the backward pass consumes it. It wins because of *how many passes you need*, which depends entirely on the shape of your problem.

Exercises

1. A function has 3 inputs and 500 outputs. Which mode should you use, and how many passes does it need?
2. Explain, in one sentence each, why symbolic differentiation fails on a 30-layer network and why finite differences fail on a 100,000-parameter one. The two failures are different in kind; say how.
3. A network has 1.5 million parameters and a forward pass takes 40 ms. How long does one gradient take by central differences? By reverse mode, assuming the $2\times$ rule? Give both in seconds.
4. Reverse mode needs the intermediate values from the forward pass, so memory scales with the depth of the network. *Gradient checkpointing* trades compute for memory by storing only every k -th intermediate and recomputing the rest. For a chain of L layers, what is the memory and the extra compute if you checkpoint every $\sqrt{L}$ layers? Why is $\sqrt{L}$ the natural choice?

The chain rule, run backwards

Reverse mode is the chain rule with the multiplications done in a particular order. This chapter makes that concrete, and then makes the crucial point: the objects being multiplied are never actually built.

A worked example, by hand

Take a small expression:

```
u = a · b
v = u + c
L = v2
```

with $a = 2$, $b = 3$, $c = 1$. The forward pass: $u = 6$, $v = 7$, $L = 49$.

Now go backwards. The rule is mechanical. At each step you hold a number — the derivative of L with respect to the value you are currently standing on — and you push it back through one operation by multiplying by that operation's local derivative.

Start at the top. $\partial L / \partial L = 1$. That is the seed, and it is always 1: the derivative of anything with respect to itself.

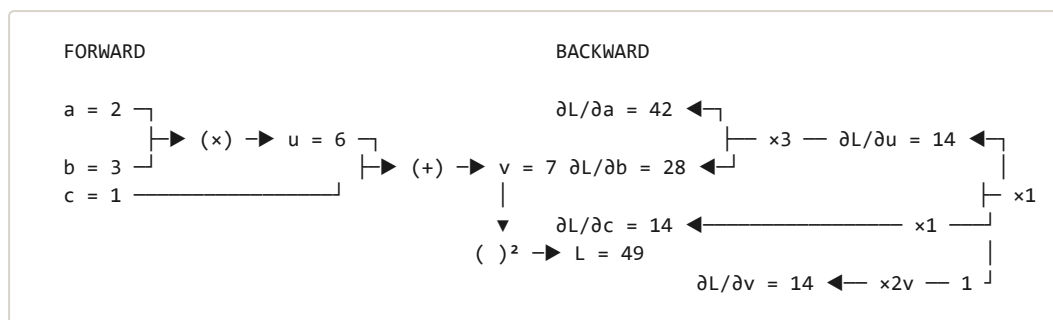
Step back through $L = v^2$. The local derivative is $\partial L / \partial v = 2v = 14$. So $\partial L / \partial v = 1 \times 14 = 14$.

Step back through $v = u + c$. Addition has local derivative 1 with respect to both operands. So $\partial L / \partial u = 14 \times 1 = 14$ and $\partial L / \partial c = 14 \times 1 = 14$.

Step back through $u = a \cdot b$. Multiplication's local derivatives are $\partial u / \partial a = b = 3$ and $\partial u / \partial b = a = 2$. So $\partial L / \partial a = 14 \times 3 = 42$ and $\partial L / \partial b = 14 \times 2 = 28$.

Check it against calculus done the ordinary way. $L = (ab + c)^2$, so $\partial L / \partial a = 2(ab + c) \cdot b = 2 \times 7 \times 3 = 42$. Correct. And $\partial L / \partial b = 2 \times 7 \times 2 = 28$. Correct.

Look at what that computation actually was. Every step: take the number you are carrying, multiply by a local derivative that depends only on values already computed in the forward pass, hand the result to the input. No global knowledge. No algebra. Each operation only needs to know its own rule.



Same graph, traversed twice. Forwards it carries values; backwards it carries derivatives, and each edge multiplies by that operation's local rule.

Vectors, and the Jacobian that must not be built

Scalars were easy because a local derivative was one number. With tensors, the general object relating an operation's output to its input is a **Jacobian**: the matrix of every output component's partial derivative with respect to every input component. If the operation maps n numbers to m numbers, its Jacobian is $m \times n$, and entry $J[i, j]$ is $\partial \text{output}_i / \partial \text{input}_j$. The chain rule in this language is a product of Jacobians, so a naive reverse mode would build each one and multiply them together.

Consider what that means for one matrix multiply, $C = A @ B$, with A and B both 512×512 . Both A and C have $512^2 = 262,144$ entries, so the Jacobian of C with respect to A is $262,144 \times 262,144 = 68.7$ billion entries. At 8 bytes each, 550 gigabytes. For one layer. Of one network. On one training step.

That is obviously impossible, and it is also completely unnecessary, because you never want the Jacobian. You want one row-vector times it.

The vector–Jacobian product

Here is the reframing that makes reverse mode work. At every point in the backward pass you are holding a vector g — the gradient of the loss with respect to this operation's output. What you need to produce is the gradient of the loss with respect to its input. By the chain rule that is:

$$g_{\text{input}} = g_{\text{output}} \cdot J$$

a vector times a matrix. The result is a vector, the same size as the input. This product is called the **vector–Jacobian product**, or VJP.

The point is that for every primitive worth having, $g \cdot J$ has a closed form that can be evaluated *without ever forming J* , and that closed form costs about as much as the forward operation did.

Matrix multiply again. The 68-billion-entry Jacobian, contracted against g , collapses to:

$$dA = g @ B^T$$

$$dB = A^T @ g$$

Two matrix multiplies of exactly the same size as the forward one. Chapter 9 derives this from the definition; for now, note the shape of the claim. The Jacobian was 550 GB. The VJP is two 512×512 matmuls. Nothing was approximated. The structure of `J` — enormously sparse and highly repetitive — was simply never written down.

Softmax is the same story: over a 50,000-token vocabulary its Jacobian is 2.5 billion entries per row, and its VJP is four elementwise operations. Chapter 9 derives both.

THE KEY IDEA

A primitive in an autodiff engine is not "an operation that knows its Jacobian". It is an operation that can answer one question: *given* `g`, *what is* `g · J`? Every gradient rule in `tensorkit/ops.py` is an answer to that question, and every one of them costs about as much as the forward operation. That is the reason the backward pass is roughly $2 \times$ the forward pass rather than $2,000,000 \times$.

COMMON MISTAKE

Because you never see the Jacobian, you never see it being wrong. A VJP with a missing term — the second half of the quotient rule, say — produces an array of exactly the right shape and entirely plausible magnitude. Nothing raises. The network trains, slightly worse, forever. Chapter 8 exists because of this.

Exercises

1. For `y = x3` at `x = 2`, do the backward pass by hand and give `∂y/∂x`.
2. `f(x) = 3x + 1` maps a length-4 vector to a length-4 vector. Write its Jacobian. Now describe its VJP in one sentence without mentioning the matrix.
3. An operation maps a `(64, 128)` tensor to a `(64, 10)` one. How many entries would its full Jacobian have? Give the number.
4. Elementwise operations have diagonal Jacobians. Explain why that makes their VJP an elementwise multiply, and then explain why `sum` — whose Jacobian is a single row of ones — has a *broadcast* as its VJP. The connection between those two facts is the subject of chapter 7.

The tape

The backward pass needs to know what the forward pass did. Something has to remember.

The obvious idea is to write the network down first, as a graph, then compile it, then run it. That is what TensorFlow 1 did, and it works, but it means the network you write is not the program you run — debugging happens in a foreign language, and `if` statements on data become special graph operators.

TensorKit does the other thing, which is what PyTorch does and what the spec calls **define-by-run**. There is no separate graph description. Every operation, as it executes, records itself. The graph is a by-product of running the forward pass, and it is thrown away and rebuilt on the next one.

What a node holds

A `Tensor` in TensorKit is an array with four extra fields:

```
class Tensor:
    data: np.ndarray           # the value from the forward pass
    grad: np.ndarray | None    # d(loss)/d(self), filled in during backward
    _prev: tuple[Tensor, ...]  # the tensors this one was produced from
    _backward: Callable[[], None] # the closure holding this node's gradient rule
    _op: str                  # name of the operation, for error messages
```

`data` and `grad` are the numbers. `_prev` and `_backward` are the tape.

`_prev` is the structure — the edges of the graph, pointing backwards from a result to the operands it came from. A tensor with `_prev == ()` was not produced by any recorded operation: it is a **leaf**, which in practice means a parameter or an input (invariant **I-LEAF**).

`_backward` is the interesting one. It is a closure, created when the operation ran, that has captured everything it needs from the forward pass. It takes no arguments and returns nothing. When called, it reads `self.grad` — the gradient that has arrived from downstream — and accumulates the appropriate contribution into each input's `.grad`.

Multiplication's closure is the canonical example. When `c = a * b` executes, it builds `c`, sets `c._prev = (a, b)`, and attaches a closure that does the equivalent of:

```
when called:
    add c.grad × (value of b) into a.grad
    add c.grad × (value of a) into b.grad
```

Two things about that. First, it captures the *values* `a.data` and `b.data`, not their gradients — the gradients do not exist yet. Second, it says *add*, not *set*, and chapter 6 is about why.

WORTH KNOWING

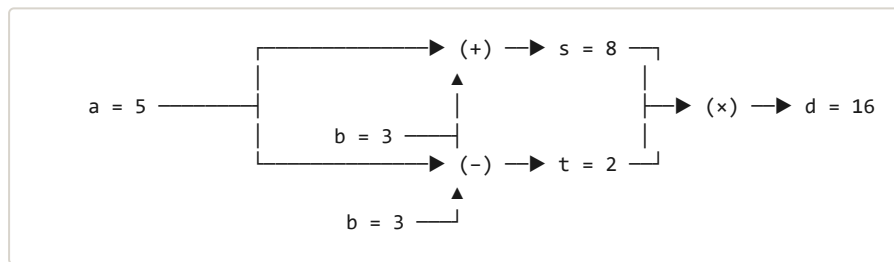
Closures are doing real work. Each rule needs different captured state: `mul` needs both operands' values, `exp` only its own output, `max` the argmax indices. A closure stores exactly that and nothing else, with no class hierarchy and no per-operation state object.

Building a graph by hand

Take the expression the milestone-1 test suite uses:

```
d = (a + b) * (a - b)      with a = 5, b = 3
```

Run it in your head as the interpreter would. Python evaluates `a + b` first. That creates a new tensor — call it `s` — with `s.data = 8`, `s._prev = (a, b)`, and an addition closure. Then `a - b` creates `t` with `t.data = 2`, `t._prev = (a, b)`, and a subtraction closure. Then the multiply creates `d` with `d.data = 16`, `d._prev = (s, t)`, and a multiplication closure.



Four nodes, five edges. Both `a` and `b` have two consumers, which is what makes this a graph rather than a tree.

Draw it on paper. The important feature is that `a` appears once as a node but twice as an operand — the arrows out of `a` fan out to two different consumers. The graph is a DAG, not a tree. This shape, a *diamond*, is where every interesting difficulty in the next three chapters comes from.

The answer, for reference: $d = a^2 - b^2$, so $\partial d / \partial a = 2a = 10$ and $\partial d / \partial b = -2b = -6$.

What is not on the tape

Two things deliberately stay off it.

Anything inside `with no_grad():` — invariant **I-NOGRAD**. Operations there still compute forward values but record no `_prev` and no `_backward`. Two places need it. Evaluation, where building a graph you will never call `backward()` on retains every activation until the garbage collector notices. And the optimiser step: `p.data -= lr * p.grad` written as tape operations would extend the graph every step, so

the tape grows without bound across training (**I-OPT-NOGRAD**). Note that `no_grad` restores the *previous* state on exit rather than unconditionally re-enabling, so an inner block does not silently turn recording back on inside an outer one.

The second thing off the tape is in-place mutation. If a tensor has already been recorded as an input, its closure captured a value. Overwriting `.data` afterwards makes the recorded rule inconsistent with the data it will be evaluated against, in a way no shape check can detect. TensorKit forbids it and raises (**I-ACYCLIC**).

Exercises

1. Draw the tape for `y = (x + 2) * x`. How many nodes? How many edges leave `x`?
2. Why does the multiply closure capture `a.data` rather than a reference to `a`? Give a case where the difference matters.
3. `Tensor.detach()` returns a new leaf sharing the same underlying buffer with no history. Give one legitimate use and one way it silently breaks a gradient.
4. Define-by-run rebuilds the graph on every forward pass, which costs Python interpreter time per operation. For a network with 500 tiny operations per step, that overhead can exceed the arithmetic. Describe what a framework could cache between steps, and what property of the program it would have to prove first.

The order of the walk

The tape is built. Every node knows its inputs and its rule. All that remains is to call every `_backward` — and the order you call them in is the whole problem.

The obvious approach, and how it fails

The obvious approach is recursion. Seed the output, call its `_backward`, and have each closure, after contributing to its inputs, recurse into them.

```
backward(node):  
    run node's local rule, pushing gradient into each input  
    for each input:  
        backward(input)
```

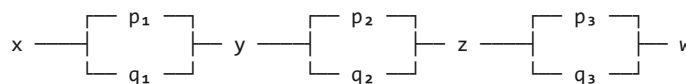
Six lines, no data structures. It is also wrong in two independent ways.

It is wrong numerically. Go back to the diamond. `a` has two consumers, `s = a + b` and `t = a - b`. When the recursion reaches `s`, it pushes `s`'s contribution into `a.grad` and then immediately recurses into `a`. But `t` has not run yet. So `a` propagates onwards carrying only *half* its gradient — the half that came through the addition. Whatever is downstream of `a` receives a partial number.

In the tiny diamond `a` is a leaf, so nothing is downstream and the bug hides. Put a diamond in the middle of a network and it does not hide. This is invariant **I-READY**:

When node `n`'s `_backward` runs, every consumer of `n` has already run, so `n.grad` is complete.

It is wrong in cost. Even if you fix the ordering, recursion visits a node once per *path* that reaches it, not once. In the single diamond, `a` is visited twice. Chain diamonds together and the path count multiplies.



paths from `w` back to `x`: $2 \times 2 \times 2 = 8$
with k diamonds: 2^k
with 30 diamonds: 1,073,741,824 visits to a graph of 121 nodes

Path count multiplies where node count adds. A residual network is structurally a chain of diamonds, so this is not a contrived shape.

A residual block — `out = x + f(x)` — is a diamond, so a thirty-block ResNet is exactly the figure above. Naive recursion on it does not give a wrong answer; it gives no answer, because it never finishes. The failure presents as a hang, which is much harder to diagnose than an exception. That is invariant **I-ONCE**:

One `backward()` call invokes each reachable node's `_backward` **exactly once**. Not zero times, not twice.

And it is wrong mechanically. Python's default recursion limit is 1,000 frames. A thirty-layer network with a few hundred operations per layer has around 9,000 nodes in a chain. The milestone-1 test builds a 10,000-node chain and asserts that the sort survives it — because a `RecursionError` in the backward pass of a model that trained fine at depth 4 is a genuinely nasty thing to debug at depth 40.

The fix: linearise first, then walk

Both correctness problems and the cost problem are solved by the same move. Do not interleave traversal with computation. Produce a complete ordering of the nodes *first*, then execute `_backward` down that list in one pass.

The ordering you need is a **topological order**: a linear sequence in which every node appears after all of its inputs. Such an order exists for any DAG, and reversing it gives you exactly the schedule I-READY demands. If every input comes before its consumer in the forward order, then in the reversed order every consumer comes before its input — so by the time you reach a node, every path into it from downstream has already deposited its contribution.

```
TOPOLOGICAL ORDER (given a root)

visited ← empty set of node identities
order   ← empty list
stack   ← [ (root, "enter") ]

while stack is not empty:
    node, phase ← pop from stack

    if phase is "enter":
        if node's identity is already in visited: continue
        mark node's identity as visited
        push (node, "exit") onto stack           # revisit after its inputs
        for each input of node:
            push (input, "enter") onto stack

    else:
        # phase is "exit"
        append node to order                     # all inputs already appended

return order    # inputs before consumers; root is last


BACKWARD (given a root and a seed)

order ← topological order of root
set root's gradient to the seed
for node in reversed(order):
    call node's backward closure
    if node is not a leaf and did not ask to keep its gradient:
        release node's gradient
```

Points worth dwelling on.

The stack carries a phase, not just a node. A node is appended to `order` only on its second visit, after everything it depends on has been appended. This is the standard iterative translation of post-order DFS, and it is the part people get wrong when converting from recursion.

Deduplicate by identity, not equality. Two distinct tensors can hold numerically equal data and must remain distinct nodes. Use `id()`, or a set of object identities.

Cycle detection belongs here. A cycle means an operation consumed its own output, which violates **I-ACYCLIC**. Without a check the walk loops forever; with one you get an exception naming the offending node. Given the choice between a hang and an exception, take the exception.

Complexity. $O(V + E)$ time and $O(V)$ space. For the 30-diamond chain: 121 nodes, 121 `_backward` calls — against 1.07 billion for the recursive version.

Seeding

One detail remains. What gradient does the root start with?

If the root is a scalar — a loss — the answer is `1.0`, because $\partial L / \partial L = 1$. That is **I-SEED**, and it is the case that matters.

If the root is not a scalar, there is no single right answer, and TensorKit raises rather than guessing. A framework could implicitly sum the output and seed with ones, which is what you want if you meant `loss.sum()`. But a caller who meant `loss.mean()` would then get gradients scaled by the element count, with no warning. An implicit semantic choice made silently on the user's behalf is worse than a `RuntimeError`. Pass an explicit seed, or reduce to a scalar first.

THE KEY IDEA

Reverse topological order is not a performance optimisation. It is the schedule that makes **I-READY** true. A node must not propagate until every consumer has contributed, and topological order is the definition of "every consumer first". **I-ONCE** — linear rather than exponential cost — comes along for free.

COMMON MISTAKE

A recursive topological sort passes every test you are likely to write by hand, because hand-written test graphs have twenty nodes. It fails on the first real model. The milestone-1 suite builds a 10,000-node chain specifically so that this failure happens on your machine in the first hour rather than on someone else's in month three.

Exercises

1. For the diamond `d = (a+b)*(a-b)`, write down a valid topological order of all four nodes. Then write down a second, different valid one.
2. Why must deduplication use `id()` rather than `==`? Construct a two-tensor example where `==` gives the wrong graph.
3. A network is a chain of 12 residual blocks. How many distinct paths run from the loss back to the input? How many `_backward` calls does the correct algorithm make on a graph of 49 nodes?
4. The pseudocode above pushes a node's inputs in whatever order `_prev` holds them. Does the resulting order depend on that? Does the *final gradient* depend on it? Justify both answers, and then name the floating-point caveat that makes the second answer not quite "no".

Accumulation

This chapter is about one character of source code: the `+` in `+=`. It is the shortest chapter and it describes the most expensive bug in the subject.

The smallest possible case

```
out = a + a
```

with `a = 3`. The forward value is 6. The derivative is 2: increase `a` by one and `out` rises by two, once through each operand.

Now watch the addition closure run. It fires once, and it does two things: push `out.grad` into the first operand's gradient, and push `out.grad` into the second operand's gradient. But both operands are the same object. So the first push writes 1 into `a.grad`, and the second push writes 1 into `a.grad`.

If the closure *assigns*, the second write overwrites the first and `a.grad` ends at 1. Wrong by a factor of two.

If the closure *accumulates*, `a.grad` ends at $1 + 1 = 2$. Correct.

That is invariant **I-ACCUM**, and it is stated in the spec as:

`_backward` **accumulates** (`+=`) into each input's `.grad`; it never assigns. A tensor consumed by k downstream operations receives k contributions and their sum is the correct total derivative.

Why the sum is the right answer

This is not a convention; it follows from the multivariable chain rule. If `a` influences `L` through several intermediate quantities `u1, u2, ..., uk`, then

$$\partial L / \partial a = \sum_i (\partial L / \partial u_i) \cdot (\partial u_i / \partial a)$$

A sum over paths. Each consumer of `a` contributes one term, and the terms add. Accumulation into `.grad` is nothing more than that sum, evaluated one term at a time as the walk reaches each consumer.

Work the diamond numerically. `d = (a+b)(a-b)` with `a = 5`, `b = 3`, so `s = 8`, `t = 2`, `d = 16`. Seed `d.grad = 1`.

step	node	closure does	a.grad	b.grad
0	—	seed <code>d.grad = 1</code>	0	0
1	<code>d = s * t</code>	<code>s.grad += 1 * t = 2</code> ; <code>t.grad += 1 * s = 8</code>	0	0
2	<code>t = a - b</code>	<code>a.grad += 8</code> ; <code>b.grad += -8</code>	8	-8
3	<code>s = a + b</code>	<code>a.grad += 2</code> ; <code>b.grad += 2</code>	10	-6

`d = a2 - b2`, so the truth is `∂d/∂a = 2a = 10` and `∂d/∂b = -2b = -6`. Both match.

Now run the same table with assignment instead. Step 3 overwrites: `a.grad = 2` and `b.grad = 2`. The gradient of `a` is wrong by 5×, and the gradient of `b` has the wrong sign. Nothing raises. The shapes are right, the dtypes are right, the magnitudes are plausible.

THE KEY IDEA

A tensor's gradient is the *sum* over every path from it to the loss. Accumulation is that sum, computed incrementally. Assignment computes the last path only, which is a number with no mathematical meaning at all — it depends on the order `_prev` happened to store its operands in.

Why this is the bug that costs a week

Every other autograd mistake announces itself. A shape error raises. A dtype error raises. A missing `no_grad` shows up as memory growth. This one does none of that.

Assignment instead of accumulation produces gradients correct in shape, correct in dtype, correct in sign for any node with a single consumer, and correct in magnitude for most of the network. Only the shared subgraphs are wrong — and in a modern architecture those are exactly the important parts: residual connections, tied embedding and output matrices, a weight reused across time steps, anything multiplied by itself.

The model trains. The loss goes down. It converges somewhere slightly worse than it should have, and the only evidence is a number you have nothing to compare against.

COMMON MISTAKE

The related trap: `zero_grad()` must set `.grad = None`, not `zeros_like(data)` — invariant **I-MOD-ZERO**. The reason is testability. With `None` meaning "no gradient has arrived", the accumulation path has an unambiguous first-arrival case, and a test can distinguish "this parameter received no gradient at all" from "this parameter's gradient is genuinely zero". Those are completely different diagnoses. `gradcheck` reports them with different messages for exactly this reason.

What TensorKit does about it

Three things, all structural rather than disciplinary.

One write path. Every gradient write goes through `Tensor.accumulate_grad`, which allocates on first arrival, adds on every subsequent arrival, and asserts I-SHAPE — that the incoming array's shape matches the tensor's. Enforcing an invariant once beats enforcing it in forty closures.

Accumulation is not reset between `backward()` calls. Calling `backward()` twice without zeroing doubles the gradients, and the spec says that is the contract, not a bug. It is what gradient accumulation across microbatches depends on: run four small batches, call `backward()` four times, step once, and you have trained on a batch four times larger than fits in memory. A test asserts the doubling, precisely so nobody "fixes" it into an implicit zeroing.

Fancy indexing gets the same treatment. `x[[0, 0, 2]]` reads index 0 twice. Its backward scatters gradient into a zero array — and `array[idx] = v` keeps only the last write while `np.add.at(array, idx, v)` accumulates. Same bug, different disguise. For an embedding table it means every frequent token learns from one occurrence per batch instead of all of them.

Exercises

1. For `y = x * x` with `x = 4`, trace the accumulation and give `x.grad`. Then give the value an assigning implementation would produce.
2. Which of these have a shared subgraph: `(a+b)*c`, `a*a`, `relu(x) + x`, `w @ x + w`? For each that does, say which node is shared.
3. A network ties its input embedding and output projection to the same $512 \times 50,000$ matrix. With assignment instead of accumulation, what fraction of that matrix's true gradient does it receive? What does that do to training?
4. `backward()` twice doubling the gradient is documented behaviour. Design a test that distinguishes it from an accumulation bug — that is, a test that passes on a correct implementation, fails on an assigning one, and does not depend on how many times `backward()` was called.

Broadcasting, and its inverse

This is the chapter people get wrong. Not because the idea is hard, but because the forward direction is so convenient that nobody thinks about it, and the backward direction has to undo something that was never explicitly done.

What broadcasting does

NumPy lets you add arrays of different shapes:

```
x = np.ones((8, 3))      # a batch of 8 rows, 3 features each
b = np.array([0.1, 0.2, 0.3]) # one bias per feature
y = x + b                # works; y.shape == (8, 3)
```

No loop, no `tile`, no copy. NumPy aligns the shapes from the right, prepends length-1 axes to the shorter one, and stretches any axis of extent 1 to match. `(3,)` becomes `(1, 3)` becomes `(8, 3)`. The same three bias values are used in all eight rows. Every neural network relies on this: a `Linear` layer is `x @ W + b`, where `x @ W` is `(batch, out)` and `b` is `(out,)`.

What the backward pass has to do

The gradient of a tensor must have the same shape as the tensor (**I-SHAPE**, no exceptions, including 0-d). But the gradient arriving at the addition is shaped like the *result*, `(8, 3)`, while `b` is shaped `(3,)`. Something has to fold `(8, 3)` down to `(3,)`.

The question is what "fold" means, and there is exactly one right answer: **sum**.

Here is why, without hand-waving. `b[0]` was used in all eight rows. Increase `b[0]` by one and all eight outputs in column 0 increase by one. The loss therefore changes by the sum of its sensitivities to those eight outputs. Sum over the broadcast axis. Not mean.

Work it on real numbers. Let the gradient arriving at the addition be:

```
g =  $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \\ 13 & 14 & 15 \\ 16 & 17 & 18 \\ 19 & 20 & 21 \\ 22 & 23 & 24 \end{bmatrix}$       (this is  $\partial \text{loss} / \partial y$ , shape (8, 3))

b.grad = column sums = [ 92, 100, 108 ]
```

Check the first: $1 + 4 + 7 + 10 + 13 + 16 + 19 + 22 = 92$. The three sums total 300, which is also the sum of all 24 entries of `g`. Not a coincidence; it is invariant **I-UB-SUMPRESERVE**:

```
unbroadcast(g, shape).sum() ≈ g.sum()
```

No gradient mass is created or destroyed by folding. It is only redistributed.

Now suppose you had written `mean` instead. You would get `[11.5, 12.5, 13.5]` — every value exactly eight times too small. The *weight* sitting next to that bias in the same layer did not broadcast, so it gets its full gradient. The bias therefore learns eight times slower than the weight beside it; with a batch of 256, 256 times slower. Nothing errors, no shape is wrong, the loss still falls. You would spend a week suspecting the learning rate.

COMMON MISTAKE

The `mean` mistake is attractive because it "keeps the scale sensible" — summing over a batch of 256 produces a bias gradient 256× larger than a per-sample one, which looks alarming. It is not alarming, it is correct. If the loss is already a mean over the batch, the 1/N is applied at the loss, once, where it belongs. Applying it again inside `unbroadcast` double-counts it.

The rule, exactly

Two distinct things happened in the forward broadcast and both must be undone. This is **I-UB-SUM** in the spec.

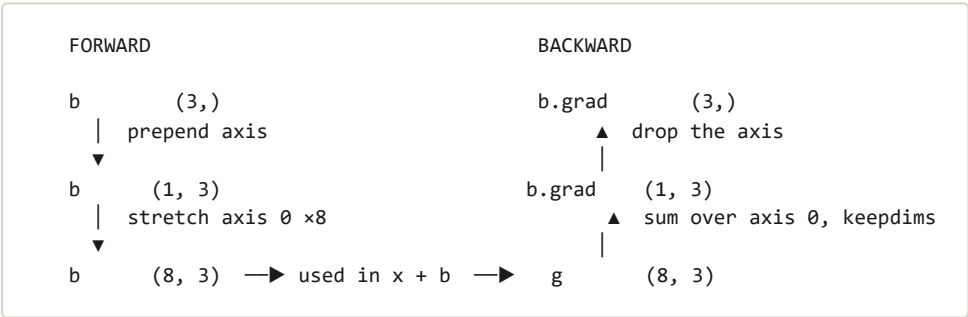
Case 1 — rank promotion. NumPy prepended axes on the left. If the gradient has rank 3 and the target rank 1, two axes were prepended. Sum over those leading axes and drop them entirely.

Case 2 — size-1 stretching. Among the remaining axes, any where the operand's extent was 1 and the result's extent was greater than 1 was stretched. Sum over those with `keepdims=True`, so the axis survives with extent 1.

Both are needed. Skipping either is a bug that **I-UB-SHAPE** — `unbroadcast(g, shape).shape == shape` for every pair NumPy would have broadcast — catches immediately, and it is property-tested over randomly generated compatible shape pairs rather than a fixed list. Here is the whole space, worked out:

gradient shape	target shape	leading axes to drop	axes to sum with keepdims	result
(4, 3)	(3,)	axis 0	none	(3,)
(4, 3)	(4, 1)	none	axis 1	(4, 1)
(4, 3)	(1, 3)	none	axis 0	(1, 3)
(2, 3, 4)	(4,)	axes 0, 1	none	(4,)
(2, 3, 4)	(1, 3, 1)	none	axes 0, 2	(1, 3, 1)
(5,)	(5,)	none	none	(5,) — identity
(2, 3)	()	axes 0, 1	none	() — a scalar

The middle rows are the ones that separate a correct implementation from one that only handles biases. $(4, 3) \rightarrow (4, 1)$ involves no rank change at all; the only clue that broadcasting happened is that the target has a 1 where the gradient has a 3. And $(2, 3, 4) \rightarrow (1, 3, 1)$ needs both cases at once, in the right order.



Every stretch in the forward direction is a sum in the backward direction, performed in the reverse order.

The deeper framing: unbroadcast is an adjoint

There is a statement that makes all of this one fact rather than a list of cases.

Broadcasting is a **linear map**: it takes a small array and produces a bigger one by copying, and that is linear in its input. Every linear map M has an **adjoint** M^T , defined by the property that for all x and g :

$$\langle Mx, g \rangle = \langle x, M^T g \rangle$$

where $\langle \cdot, \cdot \rangle$ is the sum of the elementwise product. And the VJP of a linear map is its adjoint — that is what a vector–Jacobian product reduces to when the Jacobian is constant.

So `unbroadcast` is not "the inverse of broadcasting" in the usual sense; broadcasting is not invertible, since it loses no information going up but has no unique way back down. It is the *adjoint*, and the adjoint is uniquely determined.

Check it numerically. Take `x = [1, 2]` with shape `(2,)`, broadcast to `(3, 2)`, and let

$$g = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

Left side: `broadcast(x)` is three copies of `[1, 2]`. The elementwise product with `g` summed is $(1 \cdot 1 + 2 \cdot 2) + (1 \cdot 3 + 2 \cdot 4) + (1 \cdot 5 + 2 \cdot 6) = 5 + 11 + 17 = \mathbf{33}$.

Right side: `unbroadcast(g, (2,))` is the column sums, `[9, 12]`. Then $\langle x, [9, 12] \rangle = 1 \cdot 9 + 2 \cdot 12 = \mathbf{33}$.

Equal. The milestone-2 suite property-tests this identity over randomly generated shape pairs, and it is the better test because sum preservation follows from it as a corollary — take `x` to be all ones and the identity *becomes* I-UB-SUMPRESERVE.

THE KEY IDEA

Sum and broadcast are duals. The gradient of a broadcast is a sum; the gradient of a sum is a broadcast. That is one fact with two names, and once you see it, `unbroadcast` stops being a special case and `sum`'s backward rule stops needing to be memorised.

The dual direction: sum's backward

The other direction is worth stating explicitly, because it is where the most common shape bug in a hand-rolled engine lives.

`x.sum(axis=1)` on a `(2, 3)` tensor gives shape `(2,)`. The incoming gradient is shape `(2,)`, and the output gradient must be `(2, 3)` — the incoming value broadcast back over the collapsed axis. But you cannot broadcast `(2,)` against `(2, 3)`: NumPy aligns from the right, so `(2,)` would try to match the *last* axis, of extent 3, and fail. The axis has to be *reinserted* first, `(2,)` $\rightarrow$ `(2, 1)`, and only then broadcast. With `keepdims=True` the axis was never removed, which is why the bug appears only in the `keepdims=False` path. It raises about half the time and silently produces a transposed answer the other half.

`mean` is `sum` scaled by `1/n`, where `n` is the number of elements reduced. Same reinsertion, one extra division. Getting the division wrong gives the `~1/N` ratio that `gradcheck` reports as a hint.

Exercises

1. `unbroadcast(g, shape)` where `g` has shape `(6, 1, 4)` and `shape` is `(4,)`. Which axes are dropped, which are summed with `keepdims`, and what is the result shape?
2. A batch of 32 goes through `x @ W + b`. The bias gradient sums to 96.0. What was the mean of the per-sample bias gradients along that column?
3. Write out, as numbers, `unbroadcast(g, (3, 1))` for `g = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]`. Then verify I-UB-SUMPRESERVE on your answer.
4. `matmul` broadcasts over batch dimensions: `(2, 3, 4) @ (4, 5)` gives `(2, 3, 5)`. Derive the shape of `dB` before unbroadcasting, explain why it is wrong for `B`, and say which axes must be summed to fix it. Then explain why `dA` needs no unbroadcasting in this particular case but would in `(2, 3, 4) @ (2, 4, 5)` if the first operand had batch extent 1.

Checking the gradients

You now have an engine that produces gradients, and no way to know whether they are right.

That is a worse position than it sounds. A wrong VJP has the right shape and a plausible magnitude; a wrong accumulation still trains. Nothing in the forward pass, the loss curve, or the test suite so far distinguishes a correct gradient rule from a nearly-correct one.

So bring back the method that was too slow to be an engine. As a test oracle, finite differences are perfect: independent of the code being tested, requiring no calculus, applicable to anything you can call. Milestone 3 makes them the correctness spine of the library. `gradcheck.py` is given to you fully implemented, on the principle that an oracle you wrote yourself and got subtly wrong is worse than no oracle at all.

Central differences, and why they are worth twice the cost

The forward difference is $(f(x+h) - f(x)) / h$. Expand $f(x+h)$ as a Taylor series:

$$\begin{aligned} f(x+h) &= f(x) + h f'(x) + (h^2/2) f''(x) + \dots \\ (f(x+h) - f(x)) / h &= f'(x) + (h/2) f''(x) + \dots \end{aligned}$$

The error is proportional to h . First order.

The central difference is $(f(x+h) - f(x-h)) / 2h$. Expand both:

$$\begin{aligned} f(x+h) &= f(x) + h f' + (h^2/2) f'' + (h^3/6) f''' + \dots \\ f(x-h) &= f(x) - h f' + (h^2/2) f'' - (h^3/6) f''' + \dots \end{aligned}$$

Subtract. The $f(x)$ terms cancel, and so does the $h^2/2 f''$ term — the even-order terms all cancel, which is the whole point:

$$\begin{aligned} f(x+h) - f(x-h) &= 2h f' + (h^3/3) f''' + \dots \\ (f(x+h) - f(x-h)) / 2h &= f' + (h^2/6) f''' + \dots \end{aligned}$$

Error proportional to h^2 . Second order, for one extra function evaluation. At $h = 1e-6$ that is an error term of order $1e-12$ instead of $1e-6$: six orders of magnitude for a $2\times$ cost. Nobody uses forward differences for gradient checking.

Choosing h

Now the part that determines the tolerance. Two errors pull in opposite directions.

Truncation error is what the Taylor expansion just showed: it grows as $O(h^2)$. Bigger steps approximate the limit worse.

Round-off error comes from floating point. $f(x+h)$ and $f(x-h)$ are nearly equal, each computed with a relative error of about ϵ — machine epsilon, $2.2e-16$ in float64. Subtracting two nearly equal numbers does not cancel that absolute error of $\epsilon \cdot |f|$, it exposes it, and dividing by $2h$ then amplifies it to about $\epsilon \cdot |f| / h$. It grows as $O(\epsilon/h)$. Smaller steps make it worse.

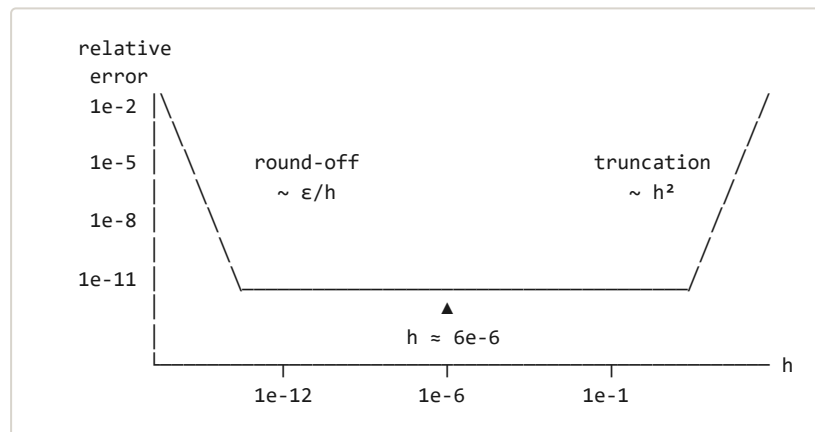
Total error, roughly:

$$E(h) \approx C \cdot h^2 + \epsilon/h$$

Differentiate and set to zero: $2Ch - \epsilon/h^2 = 0$, so $h^3 \propto \epsilon$, so:

$$h \approx \epsilon^{1/3}$$

That is where TensorKit's step size comes from. `DEFAULT_EPS` in `gradcheck.py` is literally `np.cbrt(np.finfo(np.float64).eps)`, which evaluates to **6.06e-6**.



The two error sources and their sum, on log axes. The floor is broad — anything within an order of magnitude of the optimum is fine — but both walls are steep.

Why float32 cannot be used, with the arithmetic

Put float32 through the same analysis. Its epsilon is $1.19e-7$, so the optimal step is $\text{cbrt}(1.19e-7) = 4.9e-3$ — a thousand times larger than float64's, which already tells you the truncation term will be large. The error at the optimum is roughly $\epsilon^{2/3}$: for float64, $(6.06e-6)^2 \approx 3.7e-11$; for float32, $(4.9e-3)^2 \approx 2.4e-5$.

One more amplification decides the matter. `gradcheck` differentiates `f = out.sum()`, and the round-off term scales with $|f|$, not with the derivative. If the output has 100 elements of order 1, then $|f|$ is of order 100 and the round-off term picks up that factor:

```
float32, N = 100 : 100 × 2.4e-5 / 2 ≈ 1.2e-3
float64, N = 100 : 100 × 3.7e-11 / 2 ≈ 1.8e-9
```

That is exactly the "relative error floor near 1e-3" the module docstring cites for float32, and it is why the assertable tolerance in float64 is **1e-6** — three orders of magnitude of headroom above the 1.8e-9 floor, which leaves room for larger tensors and worse-conditioned functions while still being tight enough to catch a real bug.

THE KEY IDEA

In float32 the numerical gradient is only accurate to about three digits. A tolerance loose enough to accommodate that would let a 0.1% gradient error through — and a 0.1% error is a real bug. A tolerance tight enough to be meaningful would fail on correct code. There is no usable setting. `gradcheck` therefore raises `TypeError` on a float32 input rather than checking it loosely (**I-FLOAT64**), because a false failure sends you hunting for a bug that does not exist.

Relative error, not absolute

The comparison is:

```
rel_error = |analytical - numerical| / max(|analytical|, |numerical|, 1e-12)
```

Absolute error is meaningless without scale. A gradient of 1e-8 and one of 1e8 are both perfectly ordinary — the first in a saturated sigmoid, the second in an unnormalised loss. An absolute tolerance of 1e-6 would pass everything in the first case and fail everything in the second. The `1e-12` floor in the denominator handles both gradients being legitimately zero.

Reading a failure

When `gradcheck` fails it reports the maximum relative and absolute error and the index of the worst element with both values printed. The index matters: "some gradient is wrong" and "the gradient at `[3, 7]` is wrong while its neighbours are fine" send you to different places.

The report also prints ratio hints, worth memorising because they diagnose the bug almost every time. Take the ratio of analytical to numerical at the worst element:

ratio	almost always means
≈ 2	a term is counted twice — an operand contributing through two paths that both fire, or a rule applied at both ends of a shared edge
$\approx 1/2$	in a two-path diamond, assignment where accumulation belongs — only one path survived
$\approx 1/N$	a mean where a sum belongs, N being the reduced or broadcast extent — the <code>unbroadcast</code> averaging mistake
$\approx N$	a sum where a mean belongs — the same mistake inverted
≈ 0	the rule never fires at all; the op is not on the tape, or <code>requires_grad</code> is False, or it ran under <code>no_grad</code>
sign flip	operands swapped in a non-commutative rule — <code>dA = g @ B^T</code> written as <code>B^T @ g</code> , or a subtraction's second operand missing its minus

"No gradient at all" gets its own distinct message rather than being reported as a numerical mismatch, because "this rule never ran" points at `requires_grad` and `no_grad`, while "this number is wrong by 2×" points at the rule itself. Conflating them wastes an afternoon, and a test in the milestone-3 suite pins the distinction.

Does the oracle have teeth?

A test oracle that has never been shown to fail is not evidence. So the suite plants a bug deliberately: a `div` whose denominator gradient is halved. $d(a/b)/db = -a/b^2$; the planted version returns half of that. This is a realistic mistake — the quotient rule has two terms — and it is invisible by every other means. The forward pass is exact, the loss still decreases, one operand simply learns at the wrong rate. If `gradcheck` cannot catch it, every green result it has ever produced means nothing. That is invariant **I-ORACLE**.

The suite also asserts **I-COVERAGE**: every primitive in `torch.ops.PRIMITIVES` must have a `gradcheck` case, or the registry grows while the sweep quietly does not.

Kinks

Central differences assume the function is differentiable at the point being probed. ReLU is not differentiable at 0, and neither are `abs` or `max`.

At exactly $x = 0$, `relu(0 + h) = h` and `relu(0 - h) = 0`, so the central difference is $h / 2h = 0.5$. The analytical rule picks a convention — TensorKit uses 0 — so the check reports a relative error of 1. That is not a small inaccuracy. The numerical gradient there is *meaningless* rather than imprecise: it averages the slope on both sides of a corner, and no convention will match it.

`gradcheck` takes a `kink_free` flag. Set False, it warns when any input element lies within `10·eps` of a kink and says the inputs should be moved rather than the tolerance loosened. Test inputs for kinked operations are generated away from zero.

COMMON MISTAKE

The tempting response to a kink failure is to raise the tolerance until it passes. Do not. The tolerance is doing real work — at $1e-6$ it catches a 0.0001% error, and a genuine missing term is usually a 50% error, so there is enormous margin. Loosening it to accommodate one kinked operation disarms the check for everything else. Move the input; the point where the function is not differentiable is not a point where you have any business asserting a derivative.

Exercises

1. $f(x) = x^3$. At $x = 2$, with $h = 1e-3$, compute the forward difference and the central difference by hand to four decimal places. The true derivative is 12. Which is closer, and by how much?
2. Machine epsilon for float64 is $2.22e-16$. Compute $\epsilon^{(1/3)}$. Now compute the optimal step for a hypothetical float128 with $\epsilon = 1.9e-34$.
3. A gradcheck reports analytical 0.0625 against numerical 0.5. What is the ratio, and which of the six hints in the table does it match? What would you look at first?
4. Central differences need two evaluations per *element*, so checking a (64, 128) weight needs 16,384 forward passes. Design a cheaper check that catches most bugs with a handful of passes. (Hint: you do not have to check every element of the gradient — you can check a random projection of it. What identity lets you compare $\langle v, \nabla f \rangle$ against a single directional difference?)

The gradients worth deriving by hand

Most gradient rules are one line and no insight. `exp` returns its own output; `log` returns `1/x`; `tanh` returns `1 - out2`. Write them, gradcheck them, move on. Four are different: the derivation teaches something structural, and they are the ones an interviewer asks about.

Matmul

$C = A @ B$ where A is (m, k) , B is (k, n) , C is (m, n) . Start from the definition:

$$C[i, j] = \sum_k A[i, k] \cdot B[k, j]$$

You are given $G = \partial L / \partial C$, shape (m, n) . You want $\partial L / \partial A$, shape (m, k) .

Take one entry, $A[p, q]$. Which entries of C does it affect? Only those with $i = p$ — row p — and within that row it multiplies $B[q, j]$ in every column j . So:

$$\partial C[i, j] / \partial A[p, q] = B[q, j] \quad \text{when } i = p, \quad \text{and } 0 \text{ otherwise}$$

Sum over all of C by the chain rule, keeping only the surviving terms:

$$\begin{aligned} \partial L / \partial A[p, q] &= \sum_j G[i, j] \cdot \partial C[i, j] / \partial A[p, q] \\ &= \sum_j G[p, j] \cdot B[q, j] \end{aligned}$$

Now read that last line as an index expression. It contracts the second index of G against the second index of B . That is G times B transposed:

$$dA = G @ B^T$$

Shape check: $(m, n) @ (n, k) = (m, k)$. Matches A .

Same argument for B . $B[p, q]$ affects only column q of C , multiplying $A[i, p]$:

$$\partial L / \partial B[p, q] = \sum_i A[i, p] \cdot G[i, q] \quad \Rightarrow \quad dB = A^T @ G$$

Shape check: $(k, m) @ (m, n) = (k, n)$. Matches B .

Verify on numbers. $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$, $\text{loss} = C.\text{sum}()$, so $G = \text{ones}(2, 2)$. Then $B^T = \begin{bmatrix} 5 & 7 \\ 6 & 8 \end{bmatrix}$ and $dA = \text{ones} @ B^T = \begin{bmatrix} 11 & 15 \\ 11 & 15 \end{bmatrix}$. Sanity-check the top-left entry directly: $A[0,0]$ appears in $C[0,0]$ with coefficient $B[0,0] = 5$ and in $C[0,1]$ with coefficient

$B[0,1] = 6$. Total 11. Correct. And $dB = A^T @ \text{ones} = \begin{bmatrix} 4 & 4 \\ 6 & 6 \end{bmatrix}$: $B[0,0]$ appears in $C[0,0]$ with coefficient $A[0,0] = 1$ and in $C[1,0]$ with coefficient $A[1,0] = 3$. Total 4. Correct.

Three things follow.

The cost. Two matmuls of the same order as the forward one. That is where "the backward pass costs about 2× the forward pass" comes from — matmul dominates a network's flops, and its backward is exactly two forwards.

The transposes are not decoration. Swapping to $B^T @ G$ gives a shape error if $m \neq k$, and silently gives the wrong answer if they happen to be equal — square weight matrices are common, so this bug survives to production.

Batching, and unbroadcasting again. NumPy's `@` broadcasts over leading dimensions: $(2, 3, 4) @ (4, 5)$ produces $(2, 3, 5)$. The rule holds per batch element, so dB comes out as $(2, 4, 5)$ — one per batch element. But B is $(4, 5)$. The batch axis must be summed away, which is chapter 7's `unbroadcast` doing exactly the job it was built for. A 2-D-only matmul backward returns a gradient of the wrong rank, which either raises at the optimiser or, if the optimiser is lenient, updates a broadcast view. NanoLM's attention tensors, $(\text{batch}, \text{heads}, \text{seq}, \text{dim})$, hit this on day one.

Softmax

$$s_i = \exp(a_i) / \sum_j \exp(a_j)$$

Two problems, one numerical and one structural.

Numerical first. $\exp(1000)$ overflows to `inf` in float64 — the limit is around $\exp(709.78)$ — and `inf / inf` is `nan`. Logits of magnitude 1000 are not exotic. The fix is one line and mathematically free: subtract the per-slice maximum before exponentiating.

$$s_i = \exp(a_i - m) / \sum_j \exp(a_j - m) \quad \text{where } m = \max(a)$$

Multiply numerator and denominator by $\exp(m)$ and you recover the original, so this is the identity. But now the largest exponent is exactly 0, everything else is negative, and nothing overflows. `softmax([1000, 1001])` becomes `softmax([-1, 0]) = [0.2689, 0.7311]`. Finite. There is a test asserting exactly that.

Structurally, softmax is the first operation whose Jacobian is not diagonal. Every output depends on every input, through the shared denominator. Differentiating:

$$\partial s_i / \partial a_j = s_i (\delta_{ij} - s_j)$$

where δ_{ij} is 1 when $i = j$ and 0 otherwise. In matrix form that is $\text{diag}(s) - s s^T$: dense, $n \times n$, and for a 50,000-token vocabulary, 2.5 billion entries per row of the batch. Ten gigabytes in float32, to differentiate one softmax.

So do not build it. Contract it against `g` symbolically instead:

$$\begin{aligned}(g \cdot J)_j &= \sum_i g_i \cdot s_i (\delta_{ij} - s_j) \\ &= g_j s_j - s_j \sum_i g_i s_i \\ &= s_j (g_j - \sum_i g_i s_i)\end{aligned}$$

In array notation, over the softmax axis:

$$da = s * (g - \text{sum}(g * s, \text{axis}, \text{keepdims=True}))$$

One elementwise multiply, one sum, one subtract, one multiply. Linear in `n` instead of quadratic, and the 2.5-billion-entry object never exists.

Check it on numbers. Let `s = [0.1, 0.2, 0.7]` and `g = [1, 0, 0]`. Then $\sum g \cdot s = 0.1$, and:

$$\begin{aligned}da &= [0.1 \times (1 - 0.1), \quad 0.2 \times (0 - 0.1), \quad 0.7 \times (0 - 0.1)] \\ &= [0.09, \quad -0.02, \quad -0.07]\end{aligned}$$

Those sum to zero, which is the sanity check worth remembering: adding a constant to every logit leaves softmax unchanged, so the gradient must have no component along the all-ones direction. If your softmax VJP produces a vector whose entries do not sum to (approximately) zero, it is wrong, and you can see that without running gradcheck.

Log-sum-exp, and why cross-entropy is fused

Cross-entropy is `-log(softmax(logits)[target])`. Written literally, that composition has a hole in it: for a sufficiently negative logit, softmax underflows to *exactly* 0, and `log(0)` is `-inf`, which propagates `nan` gradients through the entire batch.

The fix is to never form the softmax:

$$\begin{aligned}\log \text{softmax}(a)_i &= a_i - \text{logsumexp}(a) \\ \text{logsumexp}(a) &= m + \log \sum_j \exp(a_j - m) \quad \text{where } m = \max(a)\end{aligned}$$

Every exponent is at most 0, so nothing overflows, and the `+ m` outside restores the value exactly. `ai - logsumexp(a)` is a difference of finite numbers with no singular point anywhere. The milestone-6 suite feeds it logits of magnitude 1e4 and asserts the loss and gradients are finite.

The VJP falls out immediately, since $\partial \text{logsumexp}(a) / \partial a_j = s_j$:

$$\begin{aligned}\partial(\log \text{softmax})_i / \partial a_j &= \delta_{ij} - s_j \\ da &= g - \text{softmax}(a) \cdot \text{sum}(g, \text{axis}, \text{keepdims=True})\end{aligned}$$

Fusing that with the negative log-likelihood collapses the whole cross-entropy gradient to:

$$\partial L / \partial \text{logits} = (\text{softmax}(\text{logits}) - \text{onehot}(\text{target})) / N$$

Predicted probabilities minus the truth, averaged over the batch. That is why cross-entropy is one fused operation in every framework rather than a composition: more stable *and* cheaper.

LayerNorm, and why it has three terms

This is the spec's nominated hardest interview question. Normalise a vector $\mathbf{x}$ of length N over its last axis:

$$\begin{aligned} \mu &= (1/N) \sum_i x_i & \sigma^2 &= (1/N) \sum_i (x_i - \mu)^2 \\ \hat{x}_i &= (x_i - \mu) / s & \text{where } s &= \sqrt{\sigma^2 + \epsilon} \\ y_i &= \gamma \hat{x}_i + \beta \end{aligned}$$

Every operation so far had a local rule: output element i depended on input element i . Here every output depends on every input, through μ and again through σ^2 . So x_j reaches the loss by three routes: directly through $\hat{x}_j$'s numerator; through μ , which shifts every $\hat{x}_i$; and through σ^2 , which scales every $\hat{x}_i$. Three routes, three terms in the sum-over-paths. That is the answer to "why three terms", and it is worth being able to say before writing any algebra.

The algebra, with $\hat{g} = \partial L / \partial \hat{x}$ (the incoming gradient already multiplied by γ):

$$\begin{aligned} \partial L / \partial \sigma^2 &= -(1 / 2s^3) \cdot \sum_i \hat{g}_i (x_i - \mu) \\ \partial L / \partial \mu &= -(1 / s) \cdot \sum_i \hat{g}_i \end{aligned}$$

The second is simpler than it looks: μ also reaches the loss through σ^2 , but that path carries a factor of $\sum_i (x_i - \mu)$, which is zero by construction. Combining and substituting $\hat{x}_i = (x_i - \mu) / s$:

$$\partial L / \partial x_i = (1/s) \cdot \left[\underset{\text{direct}}{\hat{g}_i} - \underset{\text{via } \mu}{\text{mean}(\hat{g})} - \hat{x}_i \cdot \underset{\text{via } \sigma^2}{\text{mean}(\hat{g} \cdot \hat{x})} \right]$$

Two checks you can do without a computer, both of which catch real bugs:

Shift invariance. Adding a constant to every x_i leaves y unchanged, so the gradient must sum to zero over the normalised axis. Sum the bracket: $\sum \hat{g} - N \cdot \text{mean}(\hat{g}) - \text{mean}(\hat{g}\hat{x}) \cdot \sum \hat{x}$. The first two cancel, and $\sum \hat{x} = 0$ by construction. Zero. ✓

Scale invariance. Multiplying every x_i by a constant also leaves y unchanged (up to ϵ), so $\sum_i \hat{x}_i \cdot \partial L / \partial x_i$ must vanish too. That sum is $\sum \hat{g}\hat{x} - \text{mean}(\hat{g}) \cdot \sum \hat{x} - \text{mean}(\hat{g}\hat{x}) \cdot \sum \hat{x}^2$, and since $\sum \hat{x}^2 = N$, it is $N \cdot \text{mean}(\hat{g}\hat{x}) - 0 - N \cdot \text{mean}(\hat{g}\hat{x}) = 0$. ✓

The parameter gradients are ordinary by comparison, and they are chapter 7 again: γ and β are shaped like the normalised axis but broadcast over the batch, so $d\gamma = \sum_{\text{batch}} (g \cdot \hat{x})$ and $d\beta = \sum_{\text{batch}} g$, both unbroadcast back down.

THE KEY IDEA

When an operation's output element depends on more than its corresponding input element, the gradient gets one term per route. Diagonal Jacobian: one term. A shared mean: two. A shared mean and a shared variance: three. Count the shared statistics and you know how many terms to expect before you start differentiating — which is the fastest way to notice you have missed one.

COMMON MISTAKE

BatchNorm has the same three-term structure with the reduction taken over the batch instead of the features, and it brings a failure mode LayerNorm does not have: in `train()` mode an item's output depends on the *other items in the batch*. That is fine during training and unacceptable at inference, which is why BatchNorm keeps running statistics as buffers and uses them in `eval()`. Those buffers appear in `state_dict()` but must never appear in `parameters()` (**I-MOD-BUFFER**), or the optimiser will apply gradient descent to a running average.

Exercises

1. Derive `dA` for `C = A @ B` where `A` is `(3, 2)` and `B` is `(2, 4)`. State the shape of `G` and confirm the shapes work out.
2. `s = [0.5, 0.3, 0.2]`, `g = [0, 1, 0]`. Compute the softmax VJP by hand. Verify the entries sum to zero.
3. Why does subtracting the max before `exp` not change the softmax output? Prove it in one line.
4. LayerNorm has three terms. How many does `x / ||x||2` — plain L2 normalisation, no mean subtraction — have, and why? Write the VJP and check it against the scale-invariance argument above.

What you have built, and what you have not

The engine is about four hundred lines. It is worth being precise about what those four hundred lines buy, because the honest answer is both more and less than it first appears.

What you have

A general-purpose derivative machine. Not a neural network trainer — a machine that computes exact derivatives of any composition of its primitives, with respect to any of its inputs, at roughly $2\times$ the cost of evaluating the function. Neural networks are one application. Physical simulation, sensitivity analysis, and optimisation of anything expressible in NumPy are others.

Correctness you can defend. Every gradient rule in the library has been checked against central differences in float64 to a relative tolerance of $1e-6$, and the oracle has been shown to catch a planted bug. That is a stronger correctness claim than most production code carries.

Understanding that transfers unchanged. PyTorch's `autograd` is this design with better engineering. Its `Function` class is a `_backward` closure with a formal interface. Its `grad_fn` is `_prev`. Its engine does the same reverse topological walk, in C++, with a thread pool. When a PyTorch model produces a wrong gradient, the debugging is the debugging in this book.

The invariants are the transferable part. **I-ACCUM** — sum over paths, never assign. **I-ONCE** and **I-READY** — reverse topological order, exactly once per node. **I-SEED** — the scalar convention, and the refusal to guess. **I-LEAF** — free the intermediates, keep memory $O(\text{parameters})$ rather than $O(\text{activations})$. **I-NOGRAD** — do not extend the tape from the optimiser. **I-UB-SUMPRESERVE** — the fold is a sum because broadcasting duplicated the value. Those six statements are the engine. Everything else is arrays.

What you have not

Speed. TensorKit will train the target MNIST CNN past 98%, and it will be several times slower than PyTorch doing the same thing. Three reasons, all structural.

No fused kernels. `x @ W + b` followed by a ReLU is four separate NumPy calls, each writing a full intermediate array to memory and reading it back. A fused kernel does the whole chain in one pass over the data. On modern hardware the arithmetic is nearly free and the memory traffic is the cost, so fusion alone is often a $2-4\times$ win.

Python-level graph construction. Every operation allocates a `Tensor`, builds a tuple, and creates a closure. For a convolution over a large batch that overhead is invisible. For five hundred small operations per step it can exceed the arithmetic.

No GPU. NumPy is CPU-only. A matmul that takes 40 ms on a CPU core takes well under a millisecond on a modern accelerator. Nothing about the graph algorithm changes; only where the arrays live and who multiplies them.

Graph optimisation. A real framework may rewrite the recorded graph before executing it: fold constants, fuse adjacent elementwise operations, eliminate common subexpressions, reorder a chain of matmuls. `torch.compile` and XLA are entire compilers built for this. It is compiler work rather than autodiff work — but note it is *only possible* because the graph exists as data, which is the thing you built.

Higher-order gradients. `backward()` here writes into `.grad` as raw arrays, off the tape, so you cannot differentiate the backward pass. Making gradient-of-gradient work means running the backward pass through the same tape machinery as the forward pass. That is a design change, not a patch, and it is a documented non-goal.

Memory management worth the name, and **everything distributed** — allocation pooling, buffer reuse, gradient checkpointing, all-reduce, sharded optimiser state. Orthogonal problems.

Where this goes next

The surface frozen at milestone 6 — `Tensor`, `no_grad`, `Module`, `Linear`, `LayerNorm`, `Softmax`, `Dropout`, `Embedding`, `Sequential`, `Adam`, `CrossEntropyLoss` — is what the next project imports. NanoLM builds a transformer on it: byte-pair encoding, attention, causal masking, rotary embeddings, all in terms of the primitives here. If it needs a new one, that is a TensorKit change with its own gradcheck test, not a special case bolted on the side.

That is the real measure of whether this engine is finished. Not that MNIST hits 98% — that a different model, written by someone who has not read the engine's source, trains correctly on it the first time. Autodiff is infrastructure. It works when nobody has to think about it.

THE KEY IDEA

Reverse-mode automatic differentiation is a topological sort over a DAG built at runtime, plus one closed-form vector–Jacobian product per primitive, plus the discipline to add rather than assign. Everything a modern framework has beyond that is performance engineering. Valuable, hard, worth doing — but not the idea. The idea is in this book, and it is small enough to reconstruct at a whiteboard from first principles.

Exercises

1. Name the three reasons TensorKit is slower than PyTorch on the same architecture, and say which one you would attack first for a network of many small operations.
2. Higher-order gradients require the backward pass to be recorded on the tape. Explain what `accumulate_grad` would have to do differently, and what it would cost in memory.
3. A colleague reports that their network trains but plateaus 2% below the published accuracy for the same architecture. Give the three checks you would run, in order, and say what each would rule out.
4. Explain reverse-mode autodiff at a whiteboard in five minutes, from "I have a loss and some knobs" to a topological sort. No notes. If you can do it, you have finished the book.